Rúbrica 3er Parcial — Cumplimiento MACUIN

Materia: Tecnologías y Aplicaciones de Internet (TAI204)
Proyecto: MACUIN Autopartes y Distribución
Fecha de corte: 2026-04-14
Estado general: ☑ MVP — todos los criterios cubiertos al 100%

Resumen de Puntuación

Criterio	Pts	Estado	Confianza
1. Dos frontends	8	☑	Alta
2. Lógica centralizada en FastAPI	8	☑	Alta
3. API con carpetas por router	9	☑	Alta
4. Modelos SQLAlchemy	9	☑	Alta
5. Solo la API accede a la BD	8	☑	Alta
6. Todos en Docker	9	☑	Alta
7. Registro de usuarios externos	7	☑	Alta
8. Pedidos 1 a N productos	7	☑	Alta
9. Consultar pedidos por usuario	7	☑	Alta
10. CRUD usuarios internos	7	☑	Alta
11. CRUD autopartes	7	☑	Alta
12. 4+ tipos de reportes	7	☑	Alta
13. PDF + xlsx + docx	7	☑	Alta
TOTAL	**100**	☑	

Criterio 1 — Dos frontends (8 pts)

Qué pide: El sistema debe tener dos aplicaciones frontend separadas.

Cómo se cumple:

Frontend	Tecnología	Puerto host	Propósito
Flask	Python 3.13 + Jinja2	`localhost:5001`	Panel interno — personal MACUIN (ventas, almacén, logística)
Laravel	PHP 8.2 + Blade	`localhost:8080`	Portal externo — clientes B2B/B2C (talleres, refaccionarias, particulares)

Ambas son aplicaciones web completas e independientes con autenticación propia, vistas diferenciadas y roles de usuario distintos. **Ninguna contiene lógica de negocio** — solo presentación y comunicación con FastAPI vía HTTP.

Archivos de entrada principales:

App	Archivo de rutas	Archivos de vistas

App	Archivo de rutas	Archivos de vistas
Flask	`flask_app/app.py` — 28 rutas Flask (GET + POST)	`flask_app/templates/` — 15 plantillas Jinja2
Laravel	`laravel_app/routes/web.php` — 17 rutas (GET + POST + PUT)	`laravel_app/resources/views/` — 16 plantillas Blade

Rutas Flask (`flask_app/app.py`):

```
GET/POST  /login
GET       /logout
GET       /gestion-autopartes
GET/POST  /agregar-autoparte
GET/POST  /editar-autoparte/<id>
GET       /eliminar-autoparte/<id>
GET       /gestion-pedidos
GET       /detalle-pedido/<id>
POST      /pedidos/<id>/estado
GET       /gestion-usuarios-internos
GET/POST  /agregar-usuario-interno
GET/POST  /editar-usuario-interno/<id>
GET       /desactivar-usuario-interno/<id>
GET       /eliminar-usuario-interno/<id>
GET       /gestion-usuarios-externos
GET/POST  /agregar-usuario-externo
GET/POST  /editar-usuario-externo/<id>
GET       /desactivar-usuario-externo/<id>
GET       /eliminar-usuario-externo/<id>
GET       /perfil
GET       /reportes
GET       /reportes/descargar/<tipo>/<formato>
```

Rutas Laravel (`laravel_app/routes/web.php`):

```
GET/POST  /login
GET/POST  /registro
GET       /logout
GET       /forgot-password
GET       /dashboard
GET       /catalogo
GET       /catalogo/{id}
GET       /carrito
POST      /carrito/agregar
POST      /carrito/actualizar
GET/POST  /checkout
GET       /pedidos
GET       /pedido/{id}
GET       /pedido/{id}/pdf
POST      /pedido/{id}/reordenar
GET/PUT   /perfil
PUT       /perfil/password
```

Vistas Flask (`flask_app/templates/`):
`login.html`, `gestion_autopartes.html`, `agregar_autoparte.html`, `editar_autoparte.html`, `gestion_pedidos.html`, `detalle_pedido.html`, `gestion_usuarios_internos.html`,

`agregar_usuario_interno.html`, `editar_usuario_interno.html`, `gestion_usuarios_externos.html`, `agregar_usuario_externo.html`, `editar_usuario_externo.html`, `perfil.html`, `reportes.html`, `sin_permisos.html`

Vistas Laravel (`laravel_app/resources/views/`):
`login.blade.php`, `register.blade.php`, `forgot-password.blade.php`, `dashboard.blade.php`, `catalogo.blade.php`, `detalle-producto.blade.php`, `carrito.blade.php`, `checkout.blade.php`, `pedidos.blade.php`, `pedido-detalle.blade.php`, `pedido-pdf.blade.php`, `perfil.blade.php`, `welcome.blade.php` + layouts: `app.blade.php`, `navbar.blade.php`, `footer.blade.php`

Criterio 2 — Lógica centralizada en FastAPI (8 pts)

Qué pide: Toda la lógica de negocio debe residir en la API FastAPI, no en los frontends.

Cómo se cumple:

Los frontends **no tienen acceso directo a la base de datos** y no contienen ninguna lógica de negocio. Toda operación pasa por la API central:

Operación	Quién la ejecuta	Endpoint FastAPI
Validar credenciales de login	FastAPI	`POST /v1/auth/login/externo` y `/login/interno`
Verificar email único en registro	FastAPI	`POST /v1/auth/registro`
Calcular subtotal, IVA 16%, descuento por usuario	FastAPI	`POST /v1/pedidos/`
Validar stock antes de crear pedido (transacción atómica)	FastAPI	`POST /v1/pedidos/`
Descontar stock al confirmar pedido	FastAPI	`POST /v1/pedidos/`
Generar folio de pedido (`MACUIN-{AÑO}-{hex}`)	FastAPI	`POST /v1/pedidos/`
Validar descuento ≤ 100 en usuarios externos	FastAPI	`PATCH /v1/usuarios/externos/{id}`
Generar PDF, xlsx, docx en memoria	FastAPI	`GET /v1/reportes/{tipo}/{formato}`
Agregar/actualizar imagen de autoparte (upload)	FastAPI	`POST/PUT /v1/autopartes/{id}`
Bloquear login si cuenta inactiva/pendiente	FastAPI	`POST /v1/auth/login/*`

Capa de comunicación — Flask (`flask_app/api/client.py`):

```
class ApiClient:
    BASE_URL = os.getenv("API_URL", "http://localhost:8001")

    def get(path, params, auth)    → http.get(BASE_URL + path)
    def post(path, json, data, files) → http.post(...)
    def put(path, json, data, files)  → http.put(...)
    def patch(path, json)          → http.patch(...)
    def delete(path, auth)         → http.delete(...)
    def get_raw(path, auth, params) → Response sin procesar (reportes)
```

Capa de servicios — Flask (`flask_app/services/`):

- `auth.py` → `login(email, password)` → `POST /v1/auth/login/interno`
- `autopartes.py` → `listar()`, `obtener(id)`, `crear(data, file)`, `editar(id, data, file)`, `eliminar(id)`

- `pedidos.py` → `listar(estado)`, `obtener(id)`, `cambiar_estado(id, estado)`
- `usuarios.py` → `listar_internos/externos()`, `obtener_interno/externo(id)`, `crear/editar/patch/eliminar_interno/externo()`

Capa de comunicación — Laravel (`laravel_app/app/Http/Client/ApiClient.php`):
Clase estática con métodos `get()`, `post()`, `put()`, `patch()` — todos apuntan a `API_URL=http://fastapi:8000`.

Controladores Laravel (`laravel_app/app/Http/Controllers/`):

- `AuthController.php` → usa `AuthService.php`
- `CatalogoController.php` → usa `AutopartesService.php`
- `CarritoController.php` → maneja sesión carrito + llama `POST /v1/pedidos/` en checkout
- `PedidoController.php` → usa `PedidosService.php`
- `PerfilController.php` → usa `UsuariosExternosService.php`

Criterio 3 — API con carpetas estructuradas por router (9 pts)

Qué pide: La API debe estar organizada en carpetas con separación por responsabilidad.

Cómo se cumple:

```
fastapi_app/app/
├── data/                       ← Capa de persistencia (SQLAlchemy ORM)
│   ├── db.py                   ← ENGINE, sessionLocal, Base, get_db()
│   ├── ddl.sql                 ← CREATE TABLE de las 5 tablas
│   ├── dml.sql                 ← Datos de prueba (8+8+18+6 registros)
│   ├── usuario_externo.py      ← Modelo ORM tb_usuarios_externos
│   ├── usuario_interno.py      ← Modelo ORM tb_usuarios_internos
│   ├── autoparte.py            ← Modelo ORM tb_autopartes
│   ├── pedido.py               ← Modelo ORM tb_pedidos
│   └── detalle_pedido.py       ← Modelo ORM tb_detalle_pedido
├── models/                     ← Schemas Pydantic (validación de entrada)
│   ├── usuarios.py             ← Crear_UsuarioExterno
│   ├── usuarios_internos.py    ← Crear_UsuarioInterno, Actualizar_UsuarioInterno
│   ├── usuarios_externos.py    ← Crear/Actualizar/Patch_UsuarioExterno
│   ├── autopartes.py           ← Crear/Actualizar/PatchAutoparte
│   └── pedidos.py              ← ItemPedido, Crear_Pedido, CambiarEstado
├── routers/                    ← Endpoints agrupados por recurso
│   ├── auth.py                 ← prefix="/v1/auth", tags=["Autenticación"]
│   ├── usuarios_internos.py    ← prefix="/v1/usuarios/internos"
│   ├── usuarios_externos.py    ← prefix="/v1/usuarios/externos"
│   ├── autopartes.py           ← prefix="/v1/autopartes"
│   ├── pedidos.py              ← prefix="/v1/pedidos"
│   └── reportes.py             ← prefix="/v1/reportes"
├── security/                   ← Autenticación HTTPBasic
│   └── auth.py                 ← verificar_peticion() — Depends()
└── main.py                     ← FastAPI(), include_router × 6, StaticFiles, seed_db()
```

Registro de routers en `main.py`:

```
app = FastAPI(title="MACUIN API", description="...", version="1.0")
app.mount("/uploads", StaticFiles(directory="/app/uploads"), name="uploads")
app.include_router(auth.router)
app.include_router(usuarios_internos.router)
app.include_router(usuarios_externos.router)
app.include_router(autopartes.router)
```

```
app.include_router(pedidos.router)
app.include_router(reportes.router)
```

6 routers registrados, cada uno con `prefix="/v1/..."` y `tags=[...]` propio. La separación `data/ → models/ → routers/ → security/` sigue el patrón miAPI de referencia.

Criterio 4 — Modelos SQLAlchemy (9 pts)

Qué pide: La API debe usar modelos SQLAlchemy para interactuar con la base de datos.

Cómo se cumple:

Archivo (`fastapi_app/app/data/`)	Clase SQLAlchemy	Tabla PostgreSQL	Columnas clave
`usuario_externo.py`	`UsuarioExterno`	`tb_usuarios_externos`	id, nombre, apellidos, email, password, telefono, empresa, tipo_cliente, rfc, giro, calle, ciudad, estado_geo, cp, lista_precio, dias_credito, limite_credito, descuento, notas, referencia, estado
`usuario_interno.py`	`UsuarioInterno`	`tb_usuarios_internos`	id, nombre, apellidos, email, password, telefono, departamento, rol, cargo, sucursal, perm_autopartes, perm_pedidos, perm_usuarios, perm_reportes, perm_config, estado, ultima_actividad
`autoparte.py`	`Autoparte`	`tb_autopartes`	id, nombre, sku, categoria, marca, precio, precio_original, stock, stock_minimo, unidad, ubicacion, marca_vehiculo, modelo_vehiculo, descripcion, imagen, estado, activo
`pedido.py`	`Pedido`	`tb_pedidos`	id, folio, usuario_externo_id, usuario_interno_id, estado, subtotal, envio, impuestos, total, dir_calle, dir_ciudad, dir_estado, dir_cp, creado_en
`detalle_pedido.py`	`DetallePedido`	`tb_detalle_pedido`	id, pedido_id, autoparte_id, cantidad, precio_unitario, subtotal (GENERATED)

Patrón de uso en routers:

```python
# Inyección de sesión vía Depends
async def endpoint(db: Session = Depends(get_db)):
    # Consulta
    items = db.query(Modelo).filter(...).all()
    # Creación
    nuevo = Modelo(**data)
    db.add(nuevo); db.commit(); db.refresh(nuevo)
    # Actualización
    obj = db.query(Modelo).filter(Modelo.id == id).first()
    for campo, valor in data.items():
        setattr(obj, campo, valor)
    db.commit()
```

Ejemplo real (`routers/pedidos.py`):

```
query = db.query(Pedido).outerjoin(
    UsuarioExterno, Pedido.usuario_externo_id == UsuarioExterno.id
).order_by(Pedido.creado_en.desc())
```

`get_db()` en `data/db.py` usa `yield` para garantizar que la sesión se cierre siempre (patrón context manager).

Criterio 5 — Solo la API accede a la BD (8 pts)

Qué pide: Ningún frontend debe acceder directamente a la base de datos.

Cómo se cumple:

A nivel Docker (`docker-compose.yml`):

- PostgreSQL está en la red interna `monorepo-network` y **no expone su puerto al host** en producción. Solo FastAPI tiene la variable `DATABASE_URL`.
- Flask y Laravel solo tienen la variable `API_URL=http://fastapi:8000`. No tienen driver ni credencial de BD.

```yaml
postgres:
  networks: [monorepo-network]
  # Sin "ports:" en producción — inaccesible desde fuera de Docker

fastapi:
  environment:
    DATABASE_URL: postgresql://admin:123456@postgres:5432/DB_macuin

flask:
  environment:
    API_URL: http://fastapi:8000
    # Sin DATABASE_URL ni DB_* variables

laravel:
  environment:
    API_URL: http://fastapi:8000
    DB_CONNECTION: sqlite   # Solo para sesiones Laravel, no datos de negocio
```

A nivel de código:

- `flask_app/app.py` — ningún `import sqlalchemy`, ningún `import psycopg2`. Todo a través de `ApiClient`.
- `laravel_app/app/Http/` — ningún `DB::`, ningún `Model::` que apunte a PostgreSQL. Solo `Http::get/post()` a FastAPI.
- Flask usa SQLite exclusivamente para el manejo de sesiones de usuario (no para datos del negocio).

Criterio 6 — Todos los componentes en Docker (9 pts)

Qué pide: La aplicación completa debe correr en contenedores Docker.

Cómo se cumple:

`docker-compose.yml` en la raíz del repositorio orquesta 4 servicios:

Servicio	Imagen	Puerto expuesto	Healthcheck	depends_on
postgres	postgres:15	5432:5432	pg_isready -U admin -d DB_macuin (interval 10s, retries 5)	—
fastapi	macuin-fastapi (build ./fastapi_app)	8001:8000	—	postgres: condition: service_healthy
flask	macuin-flask (build ./flask_app)	5001:5000	—	fastapi
laravel	macuin-laravel (build ./laravel_app)	8080:80	—	fastapi

Características clave del compose:

- `restart: unless-stopped` en todos los servicios (resiliencia en producción)
- Volumen `postgres_data` persistente para datos de la BD entre reinicios
- Volumen bind `./fastapi_app/uploads:/app/uploads` para imágenes de autopartes (persisten entre rebuilds)
- Red compartida `monorepo-network` (bridge) — comunicación interna por nombre de servicio

Un solo comando levanta todo el sistema:

```
docker compose up --build
```

Verificado en Docker (2026-04-08):

```
✔ Container monorepo_postgres  Healthy
✔ Container monorepo_fastapi   Started
✔ Container monorepo_flask     Started
✔ Container monorepo_laravel   Started
```

Criterio 7 — Registro de usuarios externos (7 pts)

Qué pide: Debe existir un endpoint para registrar nuevos clientes.

Cómo se cumple:

Endpoint: `POST /v1/auth/registro`
Archivo FastAPI: `fastapi_app/app/routers/auth.py` → función `registro()`

```python
@router.post("/registro", status_code=201)
async def registro(usuario: Crear_UsuarioExterno, db: Session = Depends(get_db)):
    existe = db.query(UsuarioExterno).filter(UsuarioExterno.email == usuario.email).first()
    if existe:
        raise HTTPException(400, "El email ya está registrado")
    nuevo = UsuarioExterno(nombre=..., apellidos=..., email=..., password=..., telefono=...)
    db.add(nuevo); db.commit(); db.refresh(nuevo)
    return {"status": "201", "mensaje": "...", "data": {...}}
```

Schema Pydantic: `fastapi_app/app/models/usuarios.py` → `Crear_UsuarioExterno`
Campos: `nombre`, `apellidos`, `email`, `password`. Valida tipos con Pydantic.

Consumo desde Laravel:

- Vista: `laravel_app/resources/views/register.blade.php`
- Controlador: `laravel_app/app/Http/Controllers/AuthController.php` → método `registro()`
- Servicio: `laravel_app/app/Http/Services/AuthService.php` → `ApiClient::post('/v1/auth/registro', [...])`
- Middleware de sesión: `laravel_app/app/Http/Middleware/CheckSession.php` — protege rutas autenticadas

Login también implementado:

Endpoint	Archivo	Retorna
`POST /v1/auth/login/externo`	`routers/auth.py` → `login_externo()`	datos + tipo_cliente + descuento + estado
`POST /v1/auth/login/interno`	`routers/auth.py` → `login_interno()`	datos + rol + 5 permisos booleanos + estado

Ambos validan estado `activo` — cuentas inactivas/pendientes reciben `403`.

Criterio 8 — Pedidos con 1 a N productos (7 pts)

Qué pide: Se deben poder crear pedidos con múltiples productos en una sola operación.

Cómo se cumple:

Endpoint: `POST /v1/pedidos/`
Archivo: `fastapi_app/app/routers/pedidos.py`

Schema Pydantic (`fastapi_app/app/models/pedidos.py`):

```
class ItemPedido(BaseModel):
    autoparte_id: int
    cantidad: int

class Crear_Pedido(BaseModel):
    usuario_externo_id: int
    dir_calle: str
    dir_ciudad: str
    dir_estado: str
    dir_cp: str
    items: List[ItemPedido]   # ← 1 a N productos
```

Proceso en transacción atómica (todo o nada):

1. Obtener descuento del usuario externo desde `tb_usuarios_externos`
2. Para cada item: verificar `autoparte.stock >= cantidad` — si alguno falla → `400` + rollback total
3. Calcular `subtotal_item = cantidad × precio`
4. Aplicar `descuento_usuario (%)` sobre subtotal total
5. Calcular `impuestos = subtotal_con_descuento × 0.16`
6. `envio = $0.00` (política actual)
7. `total = subtotal_con_descuento + impuestos + envio`
8. Descontar stock: `autoparte.stock -= cantidad`
9. Insertar fila en `tb_pedidos` con folio `MACUIN-{AÑO}-{uuid4()[:8].upper()}`
10. Insertar N filas en `tb_detalle_pedido` (una por item)
11. `db.commit()` — si cualquier paso falla, `db.rollback()` automático

Consumo desde Laravel:

- `laravel_app/app/Http/Controllers/CarritoController.php` → método `checkout()`
- Toma el carrito de la sesión Laravel, construye el array `items`, llama `ApiClient::post('/v1/pedidos/', [...])`
- Ruta: `POST /checkout` → `CarritoController@checkout`

Criterio 9 — Consultar pedidos del usuario (7 pts)

Qué pide: Debe existir un endpoint para consultar los pedidos de un cliente específico.

Cómo se cumple:

Endpoints:

Método	Ruta	Archivo	Función	Descripción
GET	`/v1/pedidos/usuario/{usuario_id}`	`routers/pedidos.py`	`pedidos_por_usuario()`	Lista todos los pedidos del cliente con folio, estado, total, fecha
GET	`/v1/pedidos/{pedido_id}`	`routers/pedidos.py`	`detalle_pedido()`	Pedido + N líneas de detalle (nombre, SKU, imagen, cantidad, precio, subtotal)
GET	`/v1/pedidos/`	`routers/pedidos.py`	`consultar_todos()`	Lista todos los pedidos (panel interno, con `?estado=` opcional)

El detalle por ID incluye para cada línea: `nombre`, `sku`, `imagen` de la autoparte + `cantidad`, `precio_unitario`, `subtotal`.

Consumo desde Laravel:

- `laravel_app/app/Http/Controllers/PedidoController.php`
  - `index()` → `PedidosService::listarPorUsuario($id)` → `GET /v1/pedidos/usuario/{id}`
  - `show($id)` → `PedidosService::obtener($id)` → `GET /v1/pedidos/{id}`
  - `pdf($id)` → genera vista `pedido-pdf.blade.php` con datos del pedido
- Vistas: `pedidos.blade.php` (lista), `pedido-detalle.blade.php` (detalle completo)

Consumo desde Flask:

- `flask_app/app.py` → `gestion_pedidos()` → `PedidosService.listar(estado)` → `GET /v1/pedidos/?estado=`
- `flask_app/app.py` → `detalle_pedido(id)` → `PedidosService.obtener(id)` → `GET /v1/pedidos/{id}`
- Vistas: `gestion_pedidos.html`, `detalle_pedido.html`

Criterio 10 — CRUD usuarios internos (7 pts)

Qué pide: Debe existir un CRUD completo para el personal interno.

Cómo se cumple:

Router: `fastapi_app/app/routers/usuarios_internos.py` — prefix `/v1/usuarios/internos`

Método	Ruta	Función	Auth	Descripción
GET	`/v1/usuarios/internos/`	`listar()`	—	Retorna todos los empleados con sus permisos
GET	`/v1/usuarios/internos/{id}`	`obtener(id)`	—	Detalle de un empleado
POST	`/v1/usuarios/internos/`	`crear()`	—	Crear nuevo empleado (schema `Crear_UsuarioInterno`)
PUT	`/v1/usuarios/internos/{id}`	`actualizar(id)`	—	Reemplaza todos los campos (`Actualizar_UsuarioInterno`)
PATCH	`/v1/usuarios/internos/{id}`	`actualizar_parcial(id)`	—	Actualiza campos individuales (ej. `{"estado": "inactivo"}`)
DELETE	`/v1/usuarios/internos/{id}`	`eliminar(id)`	HTTPBasic	Elimina el registro permanentemente

Schema Pydantic: `fastapi_app/app/models/usuarios_internos.py`

- `Crear_UsuarioInterno`: nombre, apellidos, email, password, departamento, rol, cargo, sucursal, 5 permisos booleanos, estado
- `Actualizar_UsuarioInterno`: mismos campos, todos opcionales para PATCH

Consumo desde Flask:

- Servicio: `flask_app/services/usuarios.py` — `listar_internos()`, `obtener_interno(id)`, `crear_interno(data)`, `editar_interno(id, data)`, `patch_interno(id, campos)`, `eliminar_interno(id)`
- Rutas Flask: `/gestion-usuarios-internos`, `/agregar-usuario-interno`, `/editar-usuario-interno/<id>`, `/desactivar-usuario-interno/<id>`, `/eliminar-usuario-interno/<id>`
- Vistas: `gestion_usuarios_internos.html`, `agregar_usuario_interno.html`, `editar_usuario_interno.html`

Criterio 11 — CRUD autopartes (7 pts)

Qué pide: Debe existir un CRUD completo del catálogo de autopartes.

Cómo se cumple:

Router: `fastapi_app/app/routers/autopartes.py` — prefix `/v1/autopartes`

Método	Ruta	Función	Auth	Descripción
GET	`/v1/autopartes/`	`listar()`	—	Lista todas (`?categoria=` opcional, case-insensitive)
GET	`/v1/autopartes/{id}`	`obtener(id)`	—	Detalle de una autoparte
POST	`/v1/autopartes/`	`crear()`	—	Crear (multipart/form-data + `imagen` UploadFile opcional)

Método	Ruta	Función	Auth	Descripción
PUT	`/v1/autopartes/{id}`	`actualizar(id)`	—	Reemplazar completamente + nueva imagen opcional
PATCH	`/v1/autopartes/{id}`	`actualizar_parcial(id)`	—	Actualizar: precio, stock, stock_minimo, estado, ubicacion
DELETE	`/v1/autopartes/{id}`	`eliminar(id)`	HTTPBasic	Eliminar

Upload de imagen:

- Recibe `multipart/form-data`, campo `imagen` es `UploadFile`
- Archivo guardado en `fastapi_app/uploads/autopartes/{uuid}_{filename}`
- Volumen Docker bind: `./fastapi_app/uploads:/app/uploads`
- URL guardada en BD: `http://localhost:8001/uploads/autopartes/{uuid}_{filename}`
- Servida por `StaticFiles` montado en `/uploads` en `main.py`

Datos semilla (18 autopartes en 6 categorías):
Motor (4), Suspensión (3), Frenos (3), Eléctrico (3), Transmisión (2), Filtros (3)

Consumo desde Flask:

- Servicio: `flask_app/services/autopartes.py` — `listar()`, `obtener(id)`, `crear(data, file)`, `editar(id, data, file)`, `eliminar(id)`
- Rutas: `/gestion-autopartes`, `/agregar-autoparte`, `/editar-autoparte/<id>`, `/eliminar-autoparte/<id>`
- Vistas: `gestion_autopartes.html`, `agregar_autoparte.html`, `editar_autoparte.html`

Consumo desde Laravel:

- Servicio: `laravel_app/app/Http/Services/AutopartesService.php`
- Controlador: `CatalogoController.php` → `index()` (lista), `show($id)` (detalle)
- Vistas: `catalogo.blade.php`, `detalle-producto.blade.php`

Criterio 12 — 4+ tipos de reportes (7 pts)

Qué pide: Al menos 4 tipos de reportes accesibles mediante endpoints.

Cómo se cumple:

Router: `fastapi_app/app/routers/reportes.py` — prefix `/v1/reportes`

Tipo	Endpoint datos	Endpoint descarga	Contenido
Ventas	`GET /v1/reportes/datos/ventas`	`GET /v1/reportes/ventas/{fmt}`	Total vendido, top 5 productos, ventas por categoría/marca/mes. Filtros: `?fecha_inicio= ?fecha_fin= ?categoria=`
Inventario	`GET /v1/reportes/datos/inventario`	`GET /v1/reportes/inventario/{fmt}`	Stock actual por categoría, alertas de stock bajo (< stock_mínimo). Filtros: `?categoria= ?solo_alertas=`

Tipo	Endpoint datos	Endpoint descarga	Contenido
Pedidos	`GET /v1/reportes/datos/pedidos`	`GET /v1/reportes/pedidos/{fmt}`	Listado de pedidos, distribución por estado. Filtros: `?fecha_inicio= ?fecha_fin= ?estado=`
Usuarios	`GET /v1/reportes/datos/usuarios`	`GET /v1/reportes/usuarios/{fmt}`	Clientes externos con total de pedidos y monto acumulado comprado. Filtros: `?tipo_cliente= ?estado=`

Todos los endpoints de descarga requieren autenticación HTTPBasic (`macuin`/`123456`).
Los endpoints `/datos/*` devuelven JSON estructurado (usados por la vista Flask de reportes).

Consumo desde Flask:

- Vista de reportes interactiva: `flask_app/templates/reportes.html`
- Ruta GET `/reportes` → llama los 4 endpoints `/datos/*` y renderiza datos en pantalla con filtros
- Ruta GET `/reportes/descargar/<tipo>/<formato>` → llama `ApiClient.get_raw(f"/v1/reportes/{tipo}/{formato}", auth=("macuin","123456"))` → `send_file()` para descarga directa
- Requiere permiso `perm_reportes` en sesión

Criterio 13 — Reportes en PDF, xlsx y docx (7 pts)

Qué pide: Los reportes deben poder descargarse en tres formatos: PDF, Excel y Word.

Cómo se cumple:

El parámetro `{formato}` en cada endpoint de descarga acepta `pdf`, `xlsx` o `docx`. Cualquier otro valor devuelve `400 Bad Request`.

Formato	Librería	Características
PDF	`reportlab`	Colores MACUIN (#C41230), tablas formateadas con estilos, encabezado con nombre del reporte y fecha
xlsx	`openpyxl`	Hojas de cálculo con headers en negrita, celdas formateadas, múltiples hojas cuando aplica
docx	`python-docx`	Documento Word con tablas, colores de marca en encabezados, párrafos con título y fecha

Todos los archivos:

1. Se generan **en memoria** (`io.BytesIO`) — nunca se escriben a disco
2. Se devuelven como `StreamingResponse` con header `Content-Disposition: attachment; filename=reporte_{tipo}_{YYYYMMDD}.{fmt}`
3. El `Content-Type` correcto: `application/pdf`, `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`

Dependencias (`fastapi_app/requirements.txt`):

```
reportlab
openpyxl
```

```
python-docx
```

Flujo completo de descarga:

```
Usuario Flask → GET /reportes/descargar/ventas/pdf
  → flask_app/app.py: descargar_reporte("ventas", "pdf")
  → ApiClient.get_raw("/v1/reportes/ventas/pdf", auth=("macuin","123456"))
  → fastapi_app/routers/reportes.py: reporte_ventas("pdf")
  → io.BytesIO() + reportlab → StreamingResponse
  → Flask: send_file(BytesIO(resp.content), as_attachment=True)
  → Navegador: descarga reporte_ventas.pdf
```

Extras — Funcionalidades adicionales no requeridas por la rúbrica

E1 — Sistema de permisos granular para usuarios internos

Qué hace: Cada usuario interno tiene 5 permisos booleanos independientes que controlan qué secciones del panel Flask puede ver y usar.

Permiso	Columna BD	Secciones protegidas
`perm_autopartes`	`tb_usuarios_internos.perm_autopartes`	`/gestion-autopartes`, `/agregar-autoparte`, `/editar-autoparte/<id>`, `/eliminar-autoparte/<id>`
`perm_pedidos`	`tb_usuarios_internos.perm_pedidos`	`/gestion-pedidos`, `/detalle-pedido/<id>`, `/pedidos/<id>/estado`
`perm_usuarios`	`tb_usuarios_internos.perm_usuarios`	`/gestion-usuarios-internos`, `/gestion-usuarios-externos` + todos sus CRUD
`perm_reportes`	`tb_usuarios_internos.perm_reportes`	`/reportes`, `/reportes/descargar/<tipo>/<formato>`
`perm_config`	`tb_usuarios_internos.perm_config`	Reservado para configuración del sistema

Implementación en Flask (`flask_app/app.py`):

```python
def requiere_permiso(clave):
    r = requiere_sesion()
    if r: return r
    if not session["usuario"].get(clave):
        return render_template("sin_permisos.html"), 403
    return None

# Ejemplo de uso en cada ruta:
@app.route("/gestion-autopartes")
def gestion_autopartes():
    r = requiere_permiso("perm_autopartes")
    if r: return r
    ...
```

Vista de acceso denegado: `flask_app/templates/sin_permisos.html` — página 403 con identidad MACUIN.

Los permisos se almacenan en la sesión Flask al hacer login (`POST /v1/auth/login/interno` devuelve los 5 booleanos) y se verifican en cada request sin consultar la BD nuevamente.

E2 — Soft-delete (desactivación de cuentas sin eliminar)

Qué hace: Los botones "Desactivar cuenta" en la zona de riesgo de editar_usuario_interno y editar_usuario_externo cambian el estado a `inactivo` sin borrar el registro. Esto preserva el historial de pedidos.

Flujo:

1. Usuario pulsa "Desactivar" → modal de confirmación MACUIN aparece
2. Confirma → `GET /desactivar-usuario-interno/<id>` (Flask)
3. Flask → `patch_interno(id, {"estado": "inactivo"})` → `PATCH /v1/usuarios/internos/{id}`
4. FastAPI actualiza `tb_usuarios_internos.estado = 'inactivo'`
5. Redirect con flash "Cuenta de usuario desactivada."

Archivos:

- `flask_app/app.py` → rutas `desactivar_usuario_interno()` y `desactivar_usuario_externo()`
- `flask_app/services/usuarios.py` → `patch_interno()` y `patch_externo()`
- `fastapi_app/app/routers/usuarios_internos.py` → endpoint `PATCH /v1/usuarios/internos/{id}`
- `fastapi_app/app/routers/usuarios_externos.py` → endpoint `PATCH /v1/usuarios/externos/{id}`

E3 — Modal de confirmación con identidad visual MACUIN

Qué hace: Reemplaza el `window.confirm()` nativo del navegador (que no respeta diseño) por un modal estilizado con la identidad MACUIN: fondo oscuro semitransparente, ícono de advertencia rojo `#C41230`, tipografías Oswald/DM Sans, botón Cancelar gris y Confirmar rojo.

Implementado en: `gestion_usuarios_internos.html`, `gestion_usuarios_externos.html`, `editar_usuario_interno.html`, `editar_usuario_externo.html`

```
function macModal(url, title, msg) {
    document.getElementById('mac-modal-title').textContent = title;
    document.getElementById('mac-modal-msg').textContent   = msg;
    document.getElementById('mac-modal-confirm').href      = url;
    document.getElementById('mac-modal').style.display     = 'flex';
}
```

Se invoca con el nombre dinámico del usuario para evitar eliminaciones accidentales.

E4 — Sistema de carrito persistente en sesión (Laravel)

Qué hace: El carrito de compras se mantiene en la sesión de Laravel entre páginas. Soporta agregar, actualizar cantidades y reordenar pedidos previos.

Archivos:

- `laravel_app/app/Http/Controllers/CarritoController.php` — métodos `index()`, `agregar()`, `actualizar()`, `checkout()`, `reordenar()`
- Ruta `POST /pedido/{id}/reordenar` — carga todos los items de un pedido anterior al carrito actual

Carrito → Checkout → Pedido (flujo completo):

```
Catálogo → POST /carrito/agregar
         → sesión['carrito'] = [{id, nombre, precio, cantidad, imagen}, ...]
         → GET /checkout → muestra resumen
         → POST /checkout → CarritoController::checkout()
         → ApiClient::post('/v1/pedidos/', ['items' => [...]])
         → Limpia sesión carrito → redirect /pedidos
```

E5 — Descuento dinámico en checkout según tipo de cliente

Qué hace: El checkout calcula y muestra en tiempo real el descuento del usuario (obtenido del login) sobre el subtotal, cuando el método de pago es tarjeta o transferencia. El crédito MACUIN no aplica descuento.

Implementación: JavaScript puro en `checkout.blade.php` — no requiere llamadas adicionales a la API.

```
const aplicaDescuento = (metodo === 'tarjeta' || metodo === 'transferencia') && descuentoPct
> 0;
totalDisplay.textContent = aplicaDescuento ? fmt(totalDesc) : fmt(totalBase);
```

El descuento proviene de `$usuario['descuento']` inyectado desde el controlador (obtenido al hacer login).

E6 — Descarga de PDF de pedido individual (Laravel)

Qué hace: Desde la vista de detalle de pedido, el cliente puede descargar un PDF de su pedido con todos los detalles: folio, fecha, items, subtotal, IVA y total.

Archivos:

- Ruta: `GET /pedido/{id}/pdf` → `PedidoController::pdf($id)`
- Vista: `laravel_app/resources/views/pedido-pdf.blade.php`
- Generado del lado del cliente con `window.print()` o renderizado por el servidor

---

E7 — Vista de reportes interactiva con filtros en tiempo real (Flask)

Qué hace: La página `/reportes` del panel interno muestra los 4 reportes en pantalla (sin necesidad de descargar) con filtros de fecha, estado, categoría y tipo de cliente. Los datos se actualizan al aplicar filtros.

Archivos:

- Ruta: `flask_app/app.py` → `reportes()`
- Vista: `flask_app/templates/reportes.html`
- Llama a los 4 endpoints `/v1/reportes/datos/*` en paralelo y renderiza los datos en tablas HTML

E8 — Seed automático de base de datos al iniciar

Qué hace: Al arrancar FastAPI, si la BD está vacía, inserta automáticamente datos de prueba completos: 8 usuarios internos, 8 usuarios externos, 18 autopartes y 6 pedidos con sus líneas de detalle.

Archivo: `fastapi_app/app/main.py` → función `seed_db()`

```
def seed_db():
    db = sessionLocal()
```

```
    if db.query(UsuarioInterno).count() > 0:
        return  # Idempotente — no reinserta si ya hay datos
    db.add_all([...8 internos...])
    db.add_all([...8 externos...])
    db.add_all([...18 autopartes...])
    db.add_all([...6 pedidos + 12 líneas de detalle...])
    db.commit()

seed_db()  # Se ejecuta en startup de FastAPI
```

Garantiza que el sistema sea demostrable inmediatamente sin pasos manuales de carga de datos.

Flujos End-to-End Verificados

Flask (Panel Interno) — confirmado en Docker 2026-04-08

1. Login `artemio@macuin.mx` / `admin123` → sesión iniciada con 5 permisos ☑
2. `/gestion-autopartes` → lista las 18 autopartes desde BD ☑
3. Editar autoparte (con imagen) → `PUT /v1/autopartes/{id}` → 200 OK ☑
4. `/gestion-pedidos` → lista pedidos con estado desde BD ☑
5. `/gestion-usuarios-internos` → lista usuarios con estadísticas (total/activos/inactivos) ☑
6. `/reportes` → muestra datos de los 4 tipos de reporte ☑
7. Desactivar usuario → `PATCH estado=inactivo` → sin eliminar registro ☑

Laravel (Portal Externo) — confirmado en Docker 2026-04-08

1. Login `j.ramirez@tallercentral.mx` / `macuin123` → sesión iniciada ☑
2. `/catalogo` → lista autopartes desde BD ☑
3. `/pedidos` → `GET /v1/pedidos/usuario/1` → lista pedidos del cliente ☑
4. `/pedido/{id}` → detalle con líneas de producto ☑

Pendiente de ejecución final (código correcto, no probado end-to-end)

- Checkout completo: carrito → `POST /v1/pedidos/` → pedido creado en BD → reducción de stock
- Descarga de reportes PDF/xlsx/docx desde botones en `/reportes`
- Registro de nuevo usuario externo desde `/registro`